# Session Schedule and Activities

- The session agenda recorded:
  - 09:07: 20-minute learner presentation on bitemporal loading.
  - 09:26: Airflow introduction, followed by learner challenges.
  - 12:00–13:30: Lunch; some participants had started phase two.
  - 13:30: CI/CD and Harness.
  - 13:40–13:50: Learner presentation on Harness for Caspian.
  - 13:50: Participants focus on phase two.
  - 15:00: End of session.
- The instructor planned to check phase-two progress at 2:30 and encouraged participants to keep working until then.
- A capstone was planned for Friday:
  - Phase one is the required minimum certification phase and was estimated to take about two to three hours.
  - Phase two consists of open-ended stretch challenges.
  - Participants were to begin at 9:00 and first spend roughly 30 minutes getting aligned on the case study and requirements.
  - An additional capstone using a CG dataset was mentioned as a possible option for advanced participants, not a confirmed assignment.

# Bitemporal Loading

## Loading Strategies and Purpose

- Common loading strategies discussed included truncate-and-reload, insert/overwrite, and SCD Type 2 (SCD2).
- **SCD2** preserves historical record versions, generally by tracking effective start and end dates.
- **Bitemporal loading** extends SCD2 by tracking both business time and system time. It supports questions about what was effective in the business and what the system knew or contained at a given time—described in the session as “as

was" and "as is."

- The demonstrated configuration used a custom strategy such as `incremental_strategy = bitemporal_load_v2`, with load types including `full` and `incremental`.
- DBT does not provide this bitemporal strategy natively; the demonstrated behavior is implemented with custom macros.

## System Time and Business Time

- **System start time** records when data is processed or loaded. In the example, an Airflow variable supplied the load timestamp.
- **Business start time** represents when a record is effective for business reporting.
- The job ran around 3:00 a.m. PST for the previous business day, so the system load date could differ from the business date.
- The presenter described macro logic that derives a business start time from the load timestamp minus one millisecond. Time-zone differences may also explain apparent date discrepancies.
- Use business time when querying the state effective for a business date; use system time when querying when the data was loaded.
- A current indicator can identify the currently available record. The discussion emphasized clearly defining date semantics in data products.
- Boolean status flags can become misleading when examining multiple snapshots or bitemporal states; effective dating should be considered carefully.

## Example, Corrections, and Auditability

- The associate dimension example showed successive versions for one associate, ordered by system start time.
- Changes such as tenure advancing from one month to the next produced new record versions.
- A defect in a job-code/job-title join was fixed. The historical table retained the

prior version and recorded a new one with the corrected values.
- This was presented as an audit trail of what the system contained under the code that ran at each point in time.

“Bitemporal allows for corrections.”

## Full, Incremental, and Balance Loads

- **Full load** applies when the source sends the complete book of record each run. Reprocessing can be expensive because prior versions may be end-dated and new versions inserted.
- **Incremental load** applies when the source sends only new or changed records, such as financial transactions.
  - A business key, such as transaction ID, identifies records.
  - New keys are inserted.
  - When a known key arrives with a correction, the previous version is end-dated and a new version is inserted.
  - This supports historical investigation of corrections, including changes to transaction attribution.
- **Balance loading** was mentioned as another strategy, often associated with financial amounts. The presenter had limited direct experience with it.
- The framework supports common use cases, but not every data model fits its available strategies; custom solutions may be required.

## Implementation

- The bitemporal logic is implemented through custom macros and Jinja templates that ultimately invoke Python code.
- The implementation was described as creating a SHA-based representation of changes and stamping the resulting versions.
- The logic is abstracted through a chain of macros.
- Generated data products include bitemporal components, with the possibility of customization when needed.

# Airflow

### DAGs, Tasks, and Operators

- Airflow DAGs are written in Python.
- A **DAG** is a directed acyclic graph that defines workflow tasks and their dependencies.
- The >> operator expresses task sequencing (“and then”). For example, task one can precede task two, which can then lead to tasks three and four running in parallel.
- **Operators** connect Airflow to systems or services and provide tasks. Examples discussed included DBT, databases, Google Docs, Slack, and Teams.
- DAG-level callbacks can respond to success, failure, or SLA events. A failure callback can provide a mechanism to log or otherwise handle a DAG failure.
- A **callback** is a function run when a specified event occurs.

### DBT Integration, UI, and Astro Blueprints

- Cosmos was described as an adapter between Airflow and DBT.
- Its DBT task-group operator reads a configured DBT pipeline and represents pipeline stages as Airflow tasks.
- The Airflow UI displays DAGs, task states, code, and configuration. Task status colors include green, amber, and red.
- The team uses Astro blueprints: prewritten Airflow Python code with placeholders configured through YAML.
- The speaker estimated that blueprints cover more than 90% of data-product cases. Custom Airflow requirements may be addressed by creating a reusable organizational blueprint.
- The YAML configuration was characterized as relatively simple once the Python and Airflow concepts are understood.

## CI/CD and Deployment

### Definition and Purpose

- **Continuous integration** means frequently building and integrating code during

the software development life cycle. Integration generally involves merging code from a branch.

- **Continuous deployment** means releasing code to an environment, potentially including production.
- CI/CD automates integration and deployment. Frequent integration provides faster feedback when code is incorrect or broken and helps teams maintain a shared understanding of the product.
- The discussion also framed CI/CD as improving feedback and observability across the development lifecycle.

## Trade-offs

- Automation can be fragile or produce unintended outcomes; a human approval step is ineffective if approvals are merely automatic.
- Greater automation can contribute to correlated failures when many systems depend on shared infrastructure.
- Automated processes require ongoing maintenance and may introduce accountability, auditing, and approval requirements.
- CI/CD has an operational and maintenance burden; it is not free.

## Git, Harness, and GitOps

- Data-product code is stored and versioned in Git; this was contrasted with notebooks in Databricks.
- Harness was identified as the organization’s CI/CD system and described as approximately GitOps-based.
- In the described workflow, GitHub events—such as pull-request submission and approval—can trigger integration or deployment instead of a person manually starting the process.
- This workflow lets data creators work through a repository and pull request while the deployment machinery runs in the background.
- Git history can serve as a record of changes to the development environment.
- A more complete GitOps model can use repository configuration files to

describe and update a live environment. The speaker noted that the organization's current setup uses Git events to trigger processes but does not mirror the live environment one-to-one.
- The deployment destination for DBT and Airflow was raised but not resolved; Databricks was suggested as a possible DBT destination.

## Harness Demonstration

- The presenter demonstrated the Harness interface using the Caspian environment and explained how to locate a repository's deployment pipeline.
- **SQA deployment**:
  - A feature branch can be deployed for testing without a pull request.
  - Select the pipeline's Run option, apply the SQA input set, select the correct feature branch, and run the pipeline.
  - Deployment typically takes a few minutes.
  - SQA is shared: check who made the most recent change and confirm their testing is complete before deploying, to avoid overwriting their work.
- **Production release**:
  - Production deployments use the PRD environment and are not intended for everyone to run.
  - The release requires the appropriate fixed-value and PRD build inputs; some release-specific details may vary.
  - A CRQ number is required before each production release.
  - Pipeline logs are available to review deployment activity and investigate issues. The presenter did not demonstrate troubleshooting.
- API-token-triggered deployment through an AI coding assistant was raised as a possibility and characterized as risky. No approved approach was established.
- A participant mentioned storing a token in an environment variable; the discussion did not establish a specific approved tool or process.
- A separate webhook-triggered workflow, such as deployment after merging a development branch, was discussed as an example.
- AWS Secrets Manager was referenced in response to a question about where

secrets are stored.

## Other Session Discussion

- Potential demonstrations of DataDog, Amazon S3, and Lambda functions had been mentioned, subject to time and access; they were not established as completed demonstrations in the session notes.
- Participants discussed a development-container issue. Removing development-block containers and obtaining platform-team support were raised as possible troubleshooting steps, not as a confirmed assignment.
- Participants who completed phase two were encouraged to take a break or continue with other work.